

Decomposition of Actuarial Functions under Piecewise Mortality Assumptions

Jaroslav Drobek

Overview

1

- The project addressed actuarial calculations for a life insurance setup where mortality assumptions change at predefined age thresholds.
- Standard formulas are naturally defined over one time interval under one mortality basis.
- The technical task was to reformulate these quantities so that different parts of the same interval could use different mortality tables.
- The resulting framework was intended for both premium-related calculations and reserve-related calculations.

Core Modeling Problem

2

- Let a contract-related actuarial quantity be defined on the interval $[x, x + n]$.
- In the standard setting, one mortality basis is used on the whole interval.
- Here, the interval had to be split into subintervals, for example

$$[x, x + k], \quad [x + k, x + n],$$

with different mortality assumptions on each part.

- The original quantities therefore had to be rewritten in terms of quantities attached exclusively to the corresponding subintervals.

Mathematical Foundations

- The key objects were standard actuarial functions:

$${}_nE_x, \quad \ddot{a}_{x:\overline{n}|}, \quad A_{x:\overline{n}|}^1, \quad x:\overline{n}|[1].$$

- The fundamental decomposition principle is immediate for survival-discount factors:

$${}_nE_x = {}_kE_x \cdot {}_{n-k}E_{x+k}.$$

- Similar additive decompositions hold for annuities and insurance benefits:

$$\ddot{a}_{x:\overline{n}|} = \ddot{a}_{x:\overline{k}|} + {}_kE_x \cdot \ddot{a}_{x+k:\overline{n-k}|},$$

$$A_{x:\overline{n}|}^1 = A_{x:\overline{k}|}^1 + {}_kE_x \cdot A_{x+k:\overline{n-k}|}^1$$

- The nontrivial case is the increasing insurance function:

$$(IA)_{x:\overline{n}|}^1 = (IA)_{x:\overline{k}|}^1 + {}_kE_x \cdot (IA)_{x+k:\overline{n-k}|}^1 + k \cdot {}_kE_x \cdot A_{x+k:\overline{n-k}|}^1$$

Separation Logic

- Actuarial quantities defined on $[x, x + n]$ cannot be directly evaluated under multiple mortality bases.
- The solution is to express each quantity in terms of:
 - values on $[x, x + k]$,
 - values on $[x + k, x + n]$,
 - a linking term via survival ${}_kE_x$.
- This creates a consistent bridge between subintervals governed by different mortality assumptions.
- Most functions decompose linearly, but:
 - increasing insurance requires an additional correction term,
 - this term encodes the accumulated time shift across the boundary.
- The same logic extends naturally to time-shifted quantities used in reserve calculations.

Why This Was Nontrivial

- The issue was not merely plugging in another table.
- Each actuarial function is tied to a specific interval structure, so the original formulas had to be re-expressed in a mathematically consistent way.
- The increasing insurance term required special care:
 - it does not decompose as a simple sum,
 - an additional correction term must be preserved,
 - otherwise the resulting values would be mathematically incorrect.
- This was the core mathematical part of the task.

Extension to Time-Shifted Quantities

- Reserve calculations require the same logic after policy duration t has already elapsed.
- The relevant interval becomes

$$[x + t, x + n].$$

- This leads to modified separation formulas for shifted quantities such as

$${}_{n-t}E_{x+t}, \quad {}_{x+t \ n-t} \ddot{a}, \quad A^1_{x+t \ n-t}, \quad (IA)^1_{x+t \ n-t}.$$

- In the increasing-insurance case, the correction term changes from k to $(k - t)$, which is a structurally important detail for reserve logic.

Implementation Aspects

7

- The mathematical formulas had to be translated into production-style code paths instead of relying on direct calls to standard actuarial library functions.
- In practical terms, this meant:
 - identifying which calculations assumed one mortality basis over the whole interval,
 - replacing those calls with segmented logic,
 - ensuring correct handling of premium and reserve calculations.
- The implementation also had to remain extensible when the requirement evolved from two mortality segments to three.

Engineering Challenges

8

- The task was not isolated mathematics; it had to fit into an existing codebase and runtime environment.
- Important technical issues included:
 - locating all affected actuarial function calls,
 - mapping mathematical notation to existing variable and method structures,
 - handling inconsistencies in naming and data flow,
 - debugging configuration / parameter-loading issues that influenced runtime behavior.
- This made the project a combination of actuarial modeling, formula design, and code-level integration.

What Was Actually Solved

Signature result

A consistent computational framework was built for actuarial functions whose evaluation interval crosses mortality-regime boundaries.

- The original one-table formulas were transformed into segmented formulas.
- The framework covered both pricing-related and reserve-related quantities.
- The logic remained valid when the mortality structure became more granular.
- The main value was mathematical correctness under changing mortality assumptions within one contract timeline.

Outcome

10

- A reusable decomposition approach for piecewise mortality modeling was established.
- The approach clarified which actuarial functions decompose directly and which require correction terms.
- It provided a sound bridge between actuarial theory and implementation in an existing insurance system.
- The result is best viewed as a technical solution to a structured modeling constraint rather than as a new product formula.